

Chapter 7

Fourier Transform

7.1 Introduction

In [Chapter 4](#), we focused on images in the spatial domain, i.e., the physical world. In this chapter, we will learn about the frequency domain. The process of converting an image from the spatial domain to the frequency domain provides valuable insight into the nature of the image. In some cases, an operation can be performed more efficiently in the frequency domain than in the spatial domain. In this chapter, we introduce the various aspects of the Fourier transform and its properties. We focus exclusively on filtering an image in the frequency domain. Interested readers can refer to [\[Bra78\]](#), [\[Smi07\]](#), [\[SS03\]](#), etc. for more in-depth treatment of Fourier transformation.

The French mathematician Jean Joseph Fourier developed Fourier transforms in an attempt to solve the heat equation. During the process, he recognized that a function can be expressed as infinite sums of sines and cosines of different frequencies, now known as the Fourier series. The Fourier transform is a representation in which any function can be expressed as the integral of sines and cosines multiplied with the weighted function. Also, any function represented in either Fourier series or transform can be reconstructed completely by an inverse process. This is known as inverse Fourier transform.

This result was published in 1822 in the book *La Theorie Analytique de la Chaleur*. This idea was not welcomed, as at that time mathematicians were interested in and studied regular functions. It took

over a century to recognize the importance and power of Fourier series and transforms. Since the development of the fast Fourier transform algorithm, FFT [CT65], the applications of Fourier transforms have affected several fields including remote sensing, signal processing and image processing.

In image processing, Fourier transforms are used for the following:

- Image filtering
- Image compression
- Image enhancement
- Image restoration
- Image analysis
- Image reconstruction

In this chapter we discuss image filtering and enhancement in detail. In Chapter 14, we will discuss use of the Fourier transform in the reconstruction of magnetic resonance images.

7.2 Definition of Fourier Transform

A Fourier transform of a continuous function in one variable $f(x)$ is given by the following equation:

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx \quad (7.1)$$

where $i = \sqrt{-1}$. The function $f(x)$ can be retrieved by finding the inverse Fourier transform of $F(u)$, which is given by the following equation:

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{i2\pi ux} du. \quad (7.2)$$

The Fourier transform of a one-variable discrete function, $f(x)$ for $x = 0, 1, \dots, L-1$ is given by the following equation:

$$F(u) = \frac{1}{L} \sum_{x=0}^{L-1} f(x) e^{-\frac{i2\pi ux}{L}} \quad (7.3)$$

for $u = 0, 1, 2, \dots, L-1$. Equation 7.3 is known as the discrete Fourier transform, DFT. Likewise, the inverse discrete Fourier transform, IDFT is given by the following equation:

$$f(x) = \sum_{u=0}^{L-1} F(u) e^{\frac{-i2\pi ux}{L}} \quad (7.4)$$

for $x = 0, 1, 2, \dots, L-1$. Using Euler's formula $e^{i\theta} = \cos \theta + i \sin \theta$, the above equation simplifies to

$$F(u) = \frac{1}{L} \sum_{x=0}^{L-1} f(x) \left[\cos \left(\frac{-2ux\pi}{L} \right) - i \sin \left(\frac{-2ux\pi}{L} \right) \right] \quad (7.5)$$

Now, using the fact that the cos is an even function, i.e., $\cos(-\pi) = \cos(\pi)$ and that the sin is an odd function, i.e., $\sin(-\pi) = -\sin(\pi)$, Equation 7.5 can be simplified to:

$$F(u) = \frac{1}{L} \sum_{x=0}^{L-1} f(x) \left[\cos \left(\frac{2ux\pi}{L} \right) + i \sin \left(\frac{2ux\pi}{L} \right) \right] \quad (7.6)$$

$F(u)$ has two parts; the real part constituting the cos is represented as $R(u)$ and the imaginary part constituting the sin is represented as $I(u)$. Each term of F is known as the coefficient of the Fourier transform. Since u plays a key role in determining the frequency of the coefficients of the Fourier transform, u is known as the frequency variable, while x is known as the spatial variable.

Traditionally many experts have compared the Fourier transform to a glass prism. As a glass prism splits or separates the light into various wavelengths or frequencies that form a spectrum, the Fourier transform splits or separates a function into its coefficients, which depend on the frequency. These Fourier coefficients form a Fourier spectrum in the frequency domain.

From Equation 7.6, we know that the Fourier transform is comprised of complex numbers. For computational purposes, it is convenient to represent the Fourier transform in polar form as:

$$F(u) = |F(u)|e^{-i\theta(u)} \quad (7.7)$$

where $|F(u)| = \sqrt{R^2(u) + I^2(u)}$ is called the magnitude of the Fourier transform and $\theta(u) = \tan^{-1} \left[\frac{I(u)}{R(u)} \right]$ is called the phase angle of the transform. Power, $P(u)$, is defined as the following:

$$P(u) = R^2(u) + I^2(u) = |F(u)|^2, \quad (7.8)$$

The first value in the discrete Fourier transform is obtained by setting $u = 0$ in Equation 7.3 and then summing the product over all x . Hence, $F(0)$ is nothing but the average of $f(x)$ since $e^0 = 1$. $F(0)$ has a non-zero real part while the imaginary part is zero. Other values of F can be computed in a similar manner.

Let us consider a simple example to illustrate the Fourier transform. Let $f(x)$ be a discrete function with only four values: $f(0) = 2$, $f(1) = 3$, $f(2) = 2$ and $f(3) = 1$. Note that the size of f is 4, hence $L = 4$.

$$F(0) = \frac{1}{4} \sum_{x=0}^3 f(x) = \frac{f(0) + f(1) + f(2) + f(3)}{4} = 2$$

$$\begin{aligned}
F(1) &= \frac{1}{4} \sum_{x=0}^3 f(x) \left[\cos\left(\frac{-2\pi x}{4}\right) - i \sin\left(\frac{-i2\pi x}{4}\right) \right] \\
&= \frac{1}{4} \left(f(0) \left[\cos\left(\frac{0}{4}\right) + i \sin\left(\frac{0}{4}\right) \right] + f(1) \left[\cos\left(\frac{2\pi}{4}\right) + i \sin\left(\frac{2\pi}{4}\right) \right] \right. \\
&\quad \left. + f(2) \left[\cos\left(\frac{4\pi}{4}\right) + i \sin\left(\frac{4\pi}{4}\right) \right] + f(3) \left[\cos\left(\frac{6\pi}{4}\right) + i \sin\left(\frac{6\pi}{4}\right) \right] \right) \\
&= \frac{1}{4} (2(1 + 0i) + 3(0 + 1i) + 2(-1 + 0i) + 1(0 - 1i)) \\
&= \frac{2i}{4} = \frac{i}{2}
\end{aligned}$$

Note that $F(1)$ is purely imaginary. For $u = 2$, the value of $F(2) = 0$ and for $u = 3$, the value of $F(3) = \frac{-i}{2}$. The four coefficients of the Fourier transform are $\{2, \frac{i}{2}, 0, \frac{-i}{2}\}$.

7.3 Two-Dimensional Fourier Transform

The Fourier transform for two variables is given by the following equation:

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-i2\pi(ux+vy)} dx dy \quad (7.9)$$

and the inverse Fourier transform is

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{i2\pi(ux+vy)} du dv. \quad (7.10)$$

The discrete Fourier transform of a 2D function, $f(x, y)$ with size L and K is given by the following equation:

$$F(u, v) = \frac{1}{LK} \sum_{x=0}^{L-1} \sum_{y=0}^{K-1} f(x, y) e^{-i2\pi\left(\frac{ux}{L} + \frac{vy}{K}\right)} \quad (7.11)$$

for $u = 1, 2, \dots, L - 1$ and $v = 1, 2, \dots, K - 1$. Similar to 1D Fourier transform, $f(x, y)$ can be computed from $F(u, v)$ by computing the inverse Fourier transform, given by the following equation:

$$f(x, y) = \sum_{u=0}^{L-1} \sum_{v=0}^{K-1} F(u, v) e^{i2\pi(\frac{ux}{L} + \frac{vy}{K})} \quad (7.12)$$

for $x = 1, 2, \dots, L - 1$ and $y = 1, 2, \dots, K - 1$. As in the case of 1D DFT, u and v are the frequency variables and x and y are the spatial variables. The magnitude of the Fourier transform in 2D is given by the following equation:

$$|F(u, v)| = \sqrt{R^2(u, v) + I^2(u, v)} \quad (7.13)$$

and the phase angle is given by

$$\theta(u, v) = \tan^{-1} \left[\frac{I(u, v)}{R(u, v)} \right] \quad (7.14)$$

and the power is given by

$$P(u, v) = R^2(u, v) + I^2(u, v) = |F(u, v)|^2 \quad (7.15)$$

where $R(u, v)$ and $I(u, v)$ are the real and imaginary parts of the 2D DFT.

The properties of a 2D Fourier transform are:

1. The 2D space with x and y as variables is referred to as the spatial domain and the space with u and v as variables is referred to as the frequency domain.
2. $F(0, 0)$ is the average of all pixel values in the image. It can be obtained by substituting $u = 0$ and $v = 0$ in Equation 7.11. Hence $F(0, 0)$ is the brightest pixel in the Fourier transform image.

3. The two summations are separable. Thus, summation is performed along the x or y -directions first and in the other direction later.
4. The computational complexity of DFT is N^2 . Hence a modified method called Fast Fourier Transform (FFT) is used to calculate the Fourier transform. Cooley and Tukey developed the FFT algorithm [CT65]. FFT has a complexity of $N \log N$ and hence the word “Fast” in its name.

7.3.1 Fast Fourier Transform using Python

The following is the Python function for the forward Fast Fourier transform:

```
numpy.fft.fft2(a, s=None, axes=(-2,-1))
```

Necessary arguments:

`a` is the input image as an ndarray.

Optional arguments:

`s` is a tuple of integers that represents the length of each transformed axis of the output.

The individual elements in `s`, correspond to the length of each axis in the input image.

If the length on any axis is less than the corresponding size in the input image, then the input image along that axis is cropped. If the length on any axis is greater than the corresponding size in the input image, then the input image along that axis is padded with 0s.

`axes` is an integer used to compute the FFT. If `axis` is not specified, the last two axes are used.

Returns: output is a complex ndarray.

The Python code for the forward fast Fourier transform is given below.

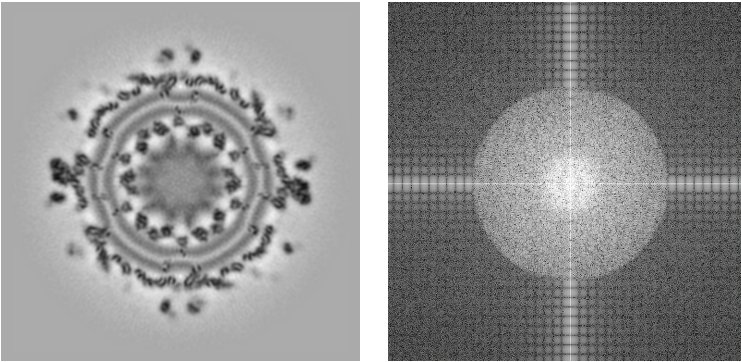
```
import scipy.fftpack as fftim
from PIL import Image

# Opening the image and converting it to grayscale.
b = Image.open('../Figures/fft1.png').convert('L')
# Performing FFT.
c = abs(fftim.fft2(b))
# Shifting the Fourier frequency image.
d = fftim.fftshift(c)
# Converting the d to floating type and saving it
# as fft1_output.raw in Figures folder.
d.astype('float').tofile('../Figures/fft1_output.raw')
```

In the above code, the image is read and converted to a gray-scale image. The Fast Fourier transform is obtained using the `fft2` function and only the absolute value is obtained for visualization. The absolute value image of FFT is then shifted, so that the center of the image is the center of the Fourier spectrum. The center pixel corresponds to a frequency of 0 in both directions. Finally, the shifted image is saved as a raw file for visualization purposes.

The image in [Figure 7.1\(a\)](#) is a slice of Sindbis virus from a transmission electron microscope. The output after performing the FFT is saved as a raw file since the pixel intensities are floating values. `ImageJ` is used to obtain the logarithm of the raw image and the window level is adjusted to display the corresponding image. Finally, a snapshot of this image is shown in [Figure 7.1\(b\)](#). As discussed previously, the central pixel is the pixel with the highest intensity. This is due to the fact

that the average of all pixel values in the original image constitutes the central pixel. The central pixel is $(0,0)$, the origin. To the left of $(0,0)$ is $-u$ and to the right is $+u$. Similarly, to the top of $(0,0)$ is $+v$ and to the bottom is $-v$. The lower frequency is close to the central pixel and the higher frequency is away from the central pixel.



(a) Input for FFT.

(b) Output of FFT.

FIGURE 7.1: An example of 2D Fast Fourier transform. Original image reprinted with permission from Dr. Wei Zhang, University of Minnesota.

The Python function for inverse Fast Fourier transform is given below.

```
numpy.fft.ifft2(a, s=None, axes=(-2,-1))
```

Necessary arguments:

`a` is a complex ndarray comprising of Fourier transformed data.

Optional arguments:

`s` is a tuple of integers that represents the length of each transformed axis of the output. The individual

elements in `s`, correspond to the length of each axis in the input image. If the length on any axis is less than the corresponding size in the input image, then the input image along that axis is cropped. If the length on any axis is greater than the corresponding size in the input image, then the input image along that axis is padded with 0s.

`axes` is an integer used to compute the FFT. If `axis` is not specified, the last two axes are used.

Returns: output is a complex ndarray.

7.4 Convolution

Convolution was briefly discussed in [Chapter 4](#), “Spatial Filters,” without any mathematical underpinning. In this section, we discuss the mathematical aspects of convolution.

Convolution is a mathematical operation that expresses the integral of the overlap between two functions. A simple example is a blurred image, which is obtained by convolving an un-blurred image with a blurring function.

There are many cases of blurred images that we see in real life. A photograph of a car moving at high speed is blurred due to motion. A photograph of a star obtained from a telescope is blurred by the particles in the atmosphere. A wide-field microscope image of an object is blurred by a signal from out-of-plane. Such blurring can be modeled as a convolution operation and eliminated by the inverse process called deconvolution.

We begin the discussion with convolution in Fourier space. The convolution operation is expressed mathematically as:

$$[f * g](t) = \int_0^t f(\tau)g(t - \tau)d\tau \quad (7.16)$$

where f, g are the two functions and the $*$ (asterisk) represents convolution.

The convolution satisfies the following properties:

1. $f * g = g * f$ Commutative Property
2. $f * (g * h) = (f * g) * h$ Associative Property
3. $f * (g + h) = f * g + f * h$ Distributive Property

The convolution operation is simpler in Fourier space than in real space but depending on the size of the image and the functions used, the former can be computationally efficient. In Fourier space, convolution is performed on the whole image at once. However, in spatial domain convolution is performed by sliding the filter window on the image.

7.4.1 Convolution in Fourier Space

Let us assume that the convolution of f and g is the function h .

$$h(t) = [f * g](t). \quad (7.17)$$

If the Fourier transform of this function is H , then H is defined as

$$H = F.G \quad (7.18)$$

where F and G are the Fourier transforms of the functions f and g respectively and the $.$ (dot) represents multiplication. Thus, in Fourier space the complex operation of convolution is replaced by a simpler multiplication. The proof of this theorem is beyond the scope of this book. You can find details in most mathematical textbooks on Fourier

transform. The formula is applicable irrespective of the number of dimensions of f and g . Hence it can be applied to a 1D signal and also to 3D volume data.

7.5 Filtering in the Frequency Domain

In this section, we discuss applying various filters to an image in the Fourier space. The convolution principle stated in Equation 7.18 will be used for filtering. In lowpass filters, only low frequencies from the Fourier transform are used while high frequencies are blocked. Similarly, in highpass filters, only high frequencies from the Fourier transform are used while the low frequencies are blocked. Lowpass filters are used to smooth the image or reduce noise, whereas highpass filters are used to sharpen edges. In each case, three different filters, namely, ideal, Butterworth and Gaussian, are considered. The three filters differ in the creation of the windows used in filtering.

7.5.1 Ideal Lowpass Filter

The convolution function for the 2D ideal lowpass filter (ILPF) is given by

$$H(u, v) = \begin{cases} 1, & \text{if } d(u, v) \leq d_0 \\ 0, & \text{else} \end{cases} \quad (7.19)$$

where d_0 is a specified quantity and $d(u, v)$ is the Euclidean distance from the point (u, v) to the origin of the Fourier domain. Note that for an image of size M by N , the coordinates of the origin are $\left(\frac{M}{2}, \frac{N}{2}\right)$. So d_0 is the distance of the cutoff frequency from the origin.

For a given image, after the convolution function is defined, the ideal lowpass filter can be performed with element-by-element multiplication

of the FFT of the image and the convolution function. Then the inverse FFT is performed on the convolved function to get the output image.

The Python code for the ideal lowpass filter is given below.

```
import cv2
import numpy, math
import scipy.fftpack as fftim
from PIL import Image

# Opening the image and converting it to grayscale.
b = Image.open('../Figures/fft1.png').convert('L')
# Performing FFT.
c = fftim.fft2(b)
# Shifting the Fourier frequency image.
d = fftim.fftshift(c)

# Intializing variables for convolution function.
M = d.shape[0]
N = d.shape[1]
# H is defined and
# values in H are initialized to 1.
H = numpy.ones((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # cut-off radius

# Defining the convolution function for ILPF.
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # Euclidean distance from
        # origin is computed.
        r = math.sqrt(r1)
```

```

# Using cut-off radius to eliminate
# high frequency.
if r > d_0:
    H[i,j] = 0.0
# Converting H to an image.
H = Image.fromarray(H)
# Performing the convolution.
con = d * H
# Computing the magnitude of the inverse FFT.
e = abs(fftim.ifft2(con))
# Saving e as ilowpass_output.png in
# Figures folder .
cv2.imwrite('../Figures/ilowpass_output.png', e)

```

The image is read and its Fourier transform is determined using the `fft2` function. The Fourier spectrum is shifted to the center of the image using the `fftshift` function. A filter (H) is created by assigning a value of 1 to all pixels within a radius of d_0 and 0 otherwise. Finally, the filter (H) is convolved with the image (d) to obtain the convolved Fourier image (con). This image is inverted using `ifft2` to obtain the filtered image in spatial domain. Since high frequencies are blocked, the image 7.2(a) is blurred.

A simple image compression technique can be created using the concept of lowpass filtering. In this technique, all high-frequency data is cleared and only the low-frequency data is stored. This reduces the number of Fourier coefficients stored and consequently needs less storage space on the disk. During the process of displaying the image, an inverse Fourier transform can be obtained to convert the image to the spatial domain. Such an image will suffer from blurring, as high frequency information is not stored. A proper selection of the cut-off radius can reduce the blurring and loss of crucial data in the decompressed image.

7.5.2 Butterworth Lowpass Filter

The convolution function for the Butterworth lowpass filter (BLPF) is given below:

$$H(u, v) = \frac{1}{1 + \left(\frac{d(u, v)}{d_0}\right)^2} \quad (7.20)$$

where d_0 is the cut-off distance from the origin for the frequency and $d(u, v)$ is the Euclidean distance from the origin. In this filter, unlike the ILPF, the pixel intensity at the cut-off radius does not change rapidly.

The Python code for the Butterworth lowpass filter is given below:

```
import numpy, math
import scipy.fftpack as fftim
from PIL import Image
import cv2

# Opening the image and converting it to grayscale.
b = Image.open('../Figures/fft1.png').convert('L')
# Performing FFT.
c = fftim.fft2(b)
# Shifting the Fourier frequency image.
d = fftim.fftshift(c)
# Intializing variables for convolution function.
M = d.shape[0]
N = d.shape[1]
# H is defined and
# values in H are initialized to 1.
H = numpy.ones((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # cut-off radius
t1 = 1 # the order of BLPF
t2 = 2*t1
```

```

# Defining the convolution function for BLPF.
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # Euclidean distance from
        # origin is computed.
        r = math.sqrt(r1)
        # Using cut-off radius to
        # eliminate high frequency.
        if r > d_0:
            H[i,j] = 1/(1 + (r/d_0)**t1)

# Converting H to an image
H = Image.fromarray(H)
# Performing the convolution.
con = d * H
# Computing the magnitude of the inverse FFT.
e = abs(fftim.ifft2(con))
# Saving e.
cv2.imwrite('../Figures/blowpass_output.png', e)

```

This program is similar to the Python code used for ILPF except for the creation of the filter (H).

7.5.3 Gaussian Lowpass Filter

The convolution function for the Gaussian lowpass filter (GLPF) is given below:

$$H(u, v) = e^{\frac{-d^2(u, v)}{2d_0^2}} \quad (7.21)$$

where d_0 is the cut-off frequency and $d(u, v)$ is the Euclidean distance from origin. The filter creates a much more gradual change in intensity at the cut-off radius compared to the Butterworth lowpass filter.

The Python code for the Gaussian lowpass filter is given below.

```
import numpy, math
import cv2
import scipy.fftpack as fftim
from PIL import Image

# Opening the image and converting it to grayscale.
b = Image.open('../Figures/fft1.png').convert('L')
# Performing FFT.
c = fftim.fft2(b)
# Shifting the Fourier frequency image.
d = fftim.fftshift(c)
# Intializing variables for convolution function.
M = d.shape[0]
N = d.shape[1]
# H is defined and
# values in H are initialized to 1.
H = numpy.ones((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # cut-off radius
t1 = 2*d_0

# Defining the convolution function for GLPF
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # euclidean distance from
        # origin is computed
        r = math.sqrt(r1)
```

```

# using cut-off radius to
# eliminate high frequency
if r > d_0:
    H[i,j] = math.exp(-r**2/t1**2)

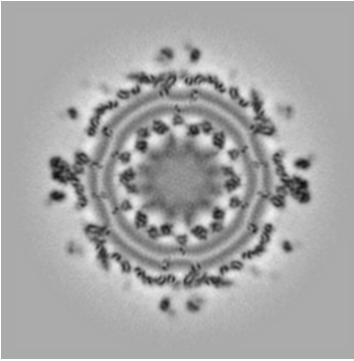
# Converting H to an image.
H = Image.fromarray(H)
# Performing the convolution.
con = d * H
# Computing the magnitude of the inverse FFT.
e = abs(fftim.ifft2(con))
# Saving the image as glowpass_output.png in
# Figures folder .
cv2.imwrite('../Figures/glowpass_output.png', e)

```

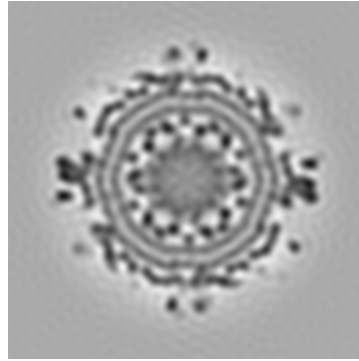
Figure 7.1 is the input image to be filtered using ILPF, BLPF and GLPF. The images in Figures 7.2(a), 7.2(b) and 7.2(c) are the outputs of ideal lowpass, Butterworth lowpass, and Gaussian lowpass filters with cut-off radius at 30. Notice how the blurriness varies in the output images. The ILPF is extremely blurred due to the sharp change in the ILPF convolution function at the cut-off radius. There are also severe ringing artifacts, the spaghetti-like structure in the background next to the foreground pixels. In BLPF, the convolution function is continuous which results in less blurring and fewer ringing artifacts compared to ILPF. Since a smoothing operator forms the GLPF convolution function, the output of GLPF is even less blurred when compared to both ILPF and BLPF.

7.5.4 Ideal Highpass Filter

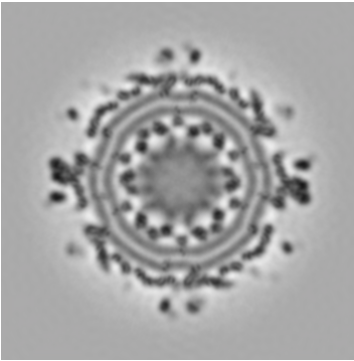
The convolution function for the 2D ideal highpass filter (IHPF) is given by



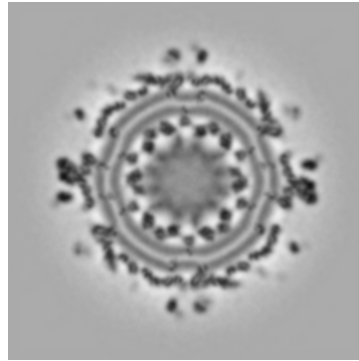
(a) Input for lowpass filters.



(b) Output of ILPF.



(c) Output of BLPF.



(d) Output of GLPF.

FIGURE 7.2: An example of lowpass filters. The input image and all the output images are displayed in the spatial domain.

$$H(u, v) = \begin{cases} 0, & \text{if } d(u, v) \leq d_0 \\ 1, & \text{else} \end{cases} \quad (7.22)$$

where d_0 is the cutoff frequency and $d(u, v)$ is the Euclidean distance from the origin.

The Python code for ideal highpass filter is given below.

```
import cv2
```

```
import numpy, math
import scipy.fftpack as fftim
from PIL import Image

# Opening the image and converting it to grayscale
a = Image.open('../Figures/endothelium.png').convert('L')
# Performing FFT.
b = fftim.fft2(a)
# shifting the Fourier frequency image
c = fftim.fftshift(b)

# initializing variables for convolution function
M = c.shape[0]
N = c.shape[1]
# H is defined and
# values in H are initialized to 1.
H = numpy.ones((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # cut-off radius

# Defining the convolution function for IHPF.
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # Euclidean distance from
# origin is computed.
        r = math.sqrt(r1)
        # Using cut-off radius to
        # eliminate low frequency.
        if 0 < r < d_0:
            H[i,j] = 0.0
# Performing the convolution.
```

```

con = c * H
# Computing the magnitude of the inverse FFT.
d = abs(fftim.ifft2(con))
# Saving the image as ihighpass_output.png in
# Figures folder.
cv2.imwrite('../Figures/ihighpass_output.png', d)

```

In this program, the filter (H) is created by assigning a pixel value of 1 to all pixels above the cut-off radius and 0 otherwise.

7.5.5 Butterworth Highpass Filter

The convolution function for the Butterworth highpass filter (BHPF) is given below:

$$H(u, v) = \frac{1}{1 + \left(\frac{d_0}{d(u, v)}\right)^{2n}} \quad (7.23)$$

where d_0 is the cut-off frequency, $d(u, v)$ is the Euclidean distance from the origin and n is the order of the BHPF.

The Python code for the BHPF is given below.

```

import cv2
import numpy, math
import scipy.misc
import scipy.fftpack as fftim
from PIL import Image

# Opening the image.
a = cv2.imread('../Figures/endothelium.png')
# Converting the image to grayscale.
b = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
# Performing FFT.
c = fftim.fft2(b)
# Shifting the Fourier frequency image.

```

```
d = fftim.fftshift(c)
# Intializing variables for convolution function.
M = d.shape[0]
N = d.shape[1]
# H is defined and
# values in H are initialized to 1.
H = numpy.ones((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # cut-off radius
t1 = 1 # the order of BHPF
t2 = 2*t1

# Defining the convolution function for BHPF.
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # Euclidean distance from
        # origin is computed.
        r = math.sqrt(r1)
        # Using cut-off radius to
        # eliminate low frequency.
        if 0 < r < d_0:
            H[i,j] = 1/(1 + (r/d_0)**t2)

# Converting H to an image.
H = Image.fromarray(H)
# performing the convolution
con = d * H
# computing the magnitude of the inverse FFT
e = abs(fftim.ifft2(con))
cv2.imwrite('../Figures/bhighpass_output.png', e)
```

7.5.6 Gaussian Highpass Filter

The convolution function for the Gaussian highpass filter (GHPF) is given below:

$$H(u, v) = 1 - e^{-\frac{d^2(u, v)}{2d_0^2}} \quad (7.24)$$

where d_0 is the cut-off frequency and $d(u, v)$ the Euclidean distance from the origin.

The Python code for the GHPF is given below.

```
import cv2
import numpy, math
import scipy.fftpack as fftim
from PIL import Image

# Opening the image and converting it to grayscale.
a = Image.open('../Figures/endothelium.png').convert('L')
# Performing FFT.
b = fftim.fft2(a)
# Shifting the Fourier frequency image.
c = fftim.fftshift(b)

# Intializing variables for convolution function.
M = c.shape[0]
N = c.shape[1]
# H is defined and values in H are initialized to 1.
H = numpy.ones((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # cut-off radius
t1 = 2*d_0
```

```

# Defining the convolution function for GHPF.
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # Euclidean distance from
        # origin is computed.
        r = math.sqrt(r1)
        # Using cut-off radius to
        # eliminate low frequency.
        if 0 < r < d_0:
            H[i,j] = 1 - math.exp(-r**2/t1**2)

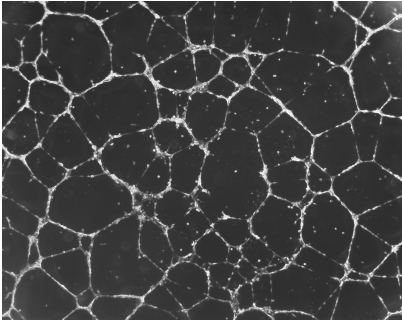
# Converting H to an image.
H = Image.fromarray(H)
# Performing the convolution.
con = c * H
# Computing the magnitude of the inverse FFT.
e = abs(fftim.ifft2(con))
# Saving the image as ghighpass_output.png in
# Figures folder.
cv2.imwrite('../Figures/ghighpass_output.png', e)

```

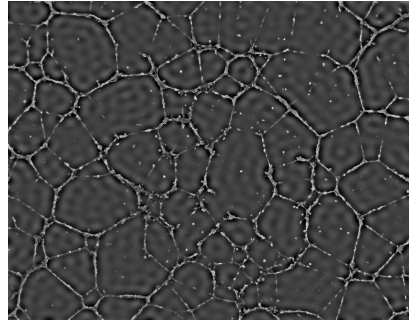
The image in [Figure 7.3\(a\)](#) is the endothelium cell. The images in [Figures 7.3\(b\)](#), [7.3\(c\)](#) and [7.3\(d\)](#) are the outputs of the IHPF, BHPF and GHPF with cut-off radius at 30. Highpass filters are used to determine edges. Notice how the edges are formed in each case.

7.5.7 Bandpass Filter

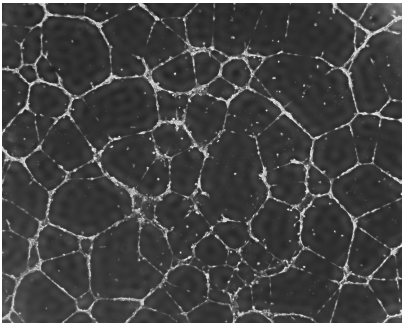
A bandpass filter, as the name indicates, allows frequency from a band or range of values. All the frequencies from outside the band are set to zero. Similar to the lowpass and highpass filters, bandpass



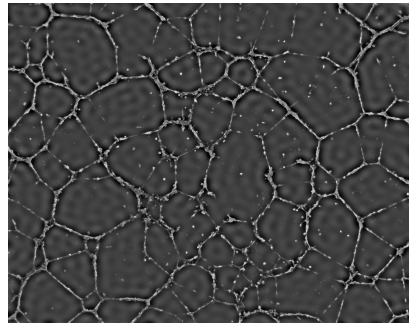
(a) Input for highpass filters.



(b) Output of the IHPF.



(c) Output of the BHPF.



(d) Output of the GHPF.

FIGURE 7.3: An example of highpass filters. The input image and all the output images are displayed in the spatial domain.

filters can be Ideal, Butterworth or Gaussian. Let us consider the ideal bandpass filter, IBPF.

The Python code for the IBPF is given below.

```
import scipy.misc
import numpy, math
import scipy.fftpack as fftim
from PIL import Image
import cv2
```

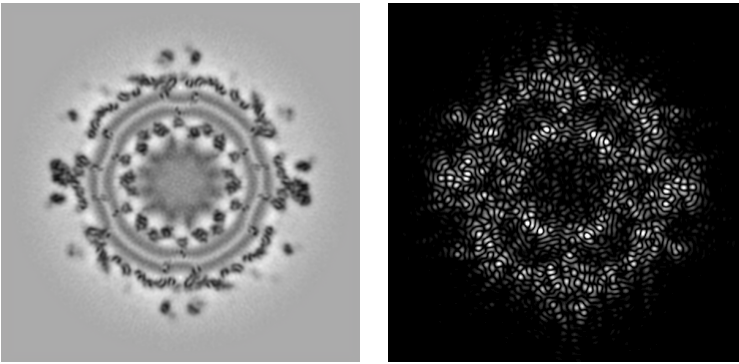
```
# Opening the image and converting it to grayscale.
b = Image.open('../Figures/fft1.png').convert('L')
# Performing FFT.
c = fftim.fft2(b)
# Shifting the Fourier frequency image .
d = fftim.fftshift(c)
# Intializing variables for convolution function.
M = d.shape[0]
N = d.shape[1]
# H is defined and
# values in H are initialized to 1.
H = numpy.zeros((M,N))
center1 = M/2
center2 = N/2
d_0 = 30.0 # minimum cut-off radius
d_1 = 50.0 # maximum cut-off radius

# Defining the convolution function for bandpass
for i in range(1,M):
    for j in range(1,N):
        r1 = (i-center1)**2+(j-center2)**2
        # Euclidean distance from
        # origin is computed.
        r = math.sqrt(r1)
        # Using min and max cut-off to create
# the band or annulus.
        if r > d_0 and r < d_1:
            H[i,j] = 1.0

# Converting H to an image.
H = Image.fromarray(H)
# Performing the convolution.
con = d * H
```

```
# Computing the magnitude of the inverse FFT.  
e = abs(fftim.iff2(con))  
# Saving the image as ibandpass_output.png.  
cv2.imwrite('../Figures/ibandpass_output.png', e)
```

The difference between this program compared to highpass or low-pass filters is in creation of the filter. In the bandpass filter, the minimum cut-off radius is set to 30 and the maximum cut-off radius is set to 50. Only intensities between 30 and 50 are passed and everything else is set to zero. [Figure 7.4\(a\)](#) is the input image and [Figure 7.4\(b\)](#) is the output image for the IBPF. Notice that the edges in the output image of the IBPF is sharp compared to the input. Similar filters can be created for Butterworth and Gaussian filters using the formula discussed earlier.



(a) Input of the IBPF.

(b) Output of the IBPF.

FIGURE 7.4: An example of IBPF. The input and the output are displayed in the spatial domain.

7.6 Summary

- Lowpass filters are used for noise reduction or smoothing. High-pass filters are used for edge enhancement or sharpening.
 - In ideal lowpass and highpass filters, Butterworth and Gaussian were considered.
 - A bandpass filter has minimum cut-off and maximum cut-off radii.
 - Convolution can be viewed as the process of combining two images. Convolution is multiplication in the Fourier domain. The inverse process is called deconvolution.
 - The Fourier transform can be used for image filtering, compression, enhancement, restoration and analysis.
-

7.7 Exercises

1. The Fourier transform is one method for converting any function as a sum of basis functions. Perform research and find at least two other such methods. Write a report on their use in image processing.

Hint: Wavelet, z -transform

2. An example for determining the Fourier coefficient was shown earlier. However, the discussion was limited to 4 coefficients. Determine the 5th coefficient assuming $f(4) = 2$.
3. The central pixel in the Fourier image is brighter compared to other pixels. Why?

4. The image in [Figure 7.2\(b\)](#) has a fuzzy structure next to the object. What is this called? What causes the artifact? Why are there fewer artifacts in BLPF and GLPF output images?
5. Consider an image of size 10,000-by-10,000 pixels that needs to be convolved with a filter of size 100-by-100. Comment about the most efficient method for convolving. Would it be convolution in the spatial domain or Fourier?